Assignment #4: PAC Learning and VC Dimension

Assigned: 10/24/2026; Due on Canvas: 11/10/26

Anca Ralescu

Overview and Instructions

This assignment has two parts:

Part	Type	Points
I	Programming (Python)	35
II	Essay	15
	Total	**50**
	Bonus (optional)	up to 6

- Submit *one* PDF containing your plots, tables, written answers to the questions in Part I, and your essay.
- Submit your code as a single `.zip` (or a Jupyter notebook) that runs from top to bottom without modification.
- Every question that asks "why", "explain" or "comment" needs a short written answer (a few sentences), not just a number or a plot.
- All written work and code must be your own or your group's work.

Notation (consistent with the lecture notes)

Examples are drawn i.i.d. from an unknown distribution P on $X \times \{-1, +1\}$. A learning machine is a family of functions $\{h_\alpha\}$. For a training set $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$,

$$R(\alpha) = \int \tfrac{1}{2}\,|h_\alpha(x) - y|\,dP(x,y), \qquad R_{\text{emp}}(\alpha) = \frac{1}{2m}\sum_{i=1}^{m} |y_i - h_\alpha(x_i)| = \frac{\#\text{training errors}}{m}.$$

With probability at least $1 - \eta$,

$$R(\alpha) \;\le\; R_{\text{emp}}(\alpha) + \underbrace{\sqrt{\frac{h\left(\ln\frac{2m}{h} + 1\right) - \ln\frac{\eta}{4}}{m}}}_{=:f_m(h)\ \text{(VC confidence)}}, \tag{1}$$

where h is the VC dimension of the family and ln is the natural logarithm. For a finite hypothesis space $\mathcal{H}$ and a learner that always returns a hypothesis consistent with the training set ($R_{\text{emp}} = 0$), the PAC sample-size bound is

$$m \;\ge\; \frac{1}{\varepsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right), \tag{2}$$

which guarantees $R \le \varepsilon$ with probability at least $1 - \delta$.

Useful facts (you may use these without proof).

- Oriented lines in $\mathbb{R}^2$ have VC dimension 3; more generally, oriented hyperplanes in $\mathbb{R}^n$ have VC dimension $n+1$.
- Four points in $\mathbb{R}^2$ can never be shattered by lines. If one point lies inside the triangle formed by the other three, the labeling "interior point $+$, the other three $-$" (and its negation) cannot be realized. If the four points form a convex quadrilateral, the "diagonal" labeling (opposite corners share a label) and its negation cannot be realized.
- Axis-aligned rectangles in $\mathbb{R}^2$ have VC dimension 4.

Part I: Programming (35 points)

Use Python 3 with `numpy`, `scipy`, `matplotlib`, and `scikit-learn`. Every plot must have labeled axes, a title, and a legend where appropriate. Fix random seeds so that results are reproducible.

P1. The VC confidence term [6 pts]

Implement $f_m(h)$ from (1) with $\eta = 0.05$.

(a) On one figure, plot $f_m(h)$ against the ratio h/m for $m \in \{10^3, 10^4, 10^5\}$ (three curves).

Range of h: h is a VC dimension, so it is a positive integer. For each m use $h = 1, 2, \ldots, m$, so that h/m runs from $1/m$ to 1. You do not need every integer: about 200 values of h, evenly spaced in h/m and rounded to integers, give a smooth curve. (The upper limit $h = m$ keeps $\ln(2m/h) > 0$; at $h/m = 1$ each curve ends near $\sqrt{1 + \ln 2} \approx 1.30$.)

(b) For each m, find numerically the ratio h/m at which $f_m(h) = 1$. For this part, treat h as a *continuous* variable so that a root finder can be used (`scipy.optimize.brentq` on the interval $h/m \in [10^{-6}, 1]$). The lecture notes report a value of about 0.37; do your results agree? Explain why the ratio barely depends on m. *Hint:* write $f_m(h)^2$ in terms of $r = h/m$ and look at which term depends on m.

(c) For $m = 10^4$, what is the largest integer VC dimension h for which the bound guarantees $R(\alpha) \le 0.1$ when $R_{\text{emp}} = 0$? When $R_{\text{emp}} = 0.05$? Comment on what this says about how restrictive the bound is.

P2. A shattering checker [9 pts]

Write a function `can_shatter(X)` that decides whether oriented hyperplanes $\operatorname{sign}(w \cdot x + b)$ shatter a point set $X \in \mathbb{R}^{m \times n}$ (m points, n coordinates). For each of the 2^m labelings $y \in \{-1, +1\}^m$, test linear separability by solving the linear-programming feasibility problem

$$\text{find } (w, b) \;\text{ such that }\; y_i(w \cdot x_i + b) \ge 1, \quad i = 1, \ldots, m,$$

with `scipy.optimize.linprog` and a zero objective. (Requiring ≥ 1 rather than > 0 loses nothing: any strict separator can be rescaled.) The set is shattered iff *every* labeling is separable. A starter skeleton:

```
import itertools
import numpy as np
from scipy.optimize import linprog
```

```python
def separable(X, y):
    """Return True if labels y in {-1,+1} are linearly separable on X."""
    m, n = X.shape
    # variables z = (w_1..w_n, b); constraint -y_i (w.x_i + b) <= -1
    A_ub = -y[:, None] * np.hstack([X, np.ones((m, 1))])
    b_ub = -np.ones(m)
    res = linprog(c=np.zeros(n + 1), A_ub=A_ub, b_ub=b_ub,
                  bounds=[(None, None)] * (n + 1), method="highs")
    return res.status == 0 # 0 = feasible solution found

def can_shatter(X):
    # TODO: loop over all labelings y in itertools.product([-1, 1], repeat=len(X))
    ...
```

(a) Verify that three non-collinear points in $\mathbb{R}^2$ (e.g. $(0,0),(1,0),(0,1)$) are shattered and that three collinear points (e.g. $(0,0),(1,0),(2,0)$) are not. Why does the second result not contradict the fact that lines in $\mathbb{R}^2$ have VC dimension 3?

(b) For each $n = 2, 3, 4, 5$, draw 1000 random sets of $n+1$ points and 1000 random sets of $n+2$ points in $\mathbb{R}^n$, with coordinates i.i.d. standard normal. Report the fraction of sets that are shattered in a table with columns n, "$n+1$ points", "$n+2$ points". Relate the results to the fact that hyperplanes in $\mathbb{R}^n$ have VC dimension $n+1$.

(c) Take one random set of 4 points in $\mathbb{R}^2$. List the labelings that are *not* separable and draw the point set with one of these labelings (color by label). Which of the two cases in the "Useful facts" above does your point set fall into? Check that the non-separable labelings are exactly the ones predicted.

(d) Write an analogous checker for axis-aligned rectangles. A labeling is realizable iff the bounding box of the positive points contains no negative point (the all-negative labeling is always realizable). Show that the four points $(\pm 1, 0), (0, \pm 1)$ are shattered, and that none of 1000 random sets of 5 points in $\mathbb{R}^2$ is shattered.

P3. PAC learning in action: rectangles and thresholds [12 pts]

Setup. Let P be uniform on the unit square $[0,1]^2$, and let the target concept be the fixed axis-aligned rectangle $c^* = [0.2, 0.7] \times [0.3, 0.6]$ (points inside are labeled $+1$, all others -1; there is no label noise). The learner returns the *tightest* axis-aligned rectangle $\hat{h}$ containing all positive training points; if there are no positive training points, it returns the empty rectangle (everything labeled -1).

(a) Explain briefly why this learner is consistent ($R_{\text{emp}} = 0$) and why $\hat{h} \subseteq c^*$. Conclude that the true error is $R = \text{area}(c^*) - \text{area}(\hat{h}) = 0.15 - \text{area}(\hat{h})$, and compute it *exactly* with this formula rather than by sampling test points.

(b) For $m \in \{10, 20, 50, 100, 200, 500, 1000\}$, run $T = 2000$ independent trials (a fresh training set each time). Plot (i) the mean true error versus m on log–log axes and (ii) the fraction of trials with $R > \varepsilon$, an estimate of $\mathbb{P}(R > \varepsilon)$, versus m for $\varepsilon = 0.05$.

(c) The classical PAC bound for this learner says that $m \geq \frac{4}{\varepsilon} \ln \frac{4}{\delta}$ examples suffice for $\mathbb{P}(R > \varepsilon) \leq \delta$. For $\varepsilon = \delta = 0.05$, compute this m and mark it on plot (ii). Then find the smallest m (to the

nearest 10, using additional values of m as needed) for which your empirical $\mathbb{P}(R > \varepsilon) \le \delta$. By what factor is the theoretical bound pessimistic?

(d) Fit a straight line to the log–log plot of mean error versus m (e.g. `np.polyfit` on $\log m$ and $\log R$). What rate $R \sim m^{-\beta}$ do you observe? Is it closer to $1/m$ or to $1/\sqrt{m}$? Which of the two is predicted for a consistent learner by (2), and which by (1)?

P4. Structural Risk Minimization versus cross-validation [8 pts]

Data. Draw $x \sim \text{Uniform}[-1, 1]$ and set $y = \text{sign}\big(\sin(3\pi x)\big)$; then flip each label independently with probability 0.1. Use $m = 200$ training points and 10^4 test points (test labels are also flipped, so the best achievable test error is 0.1).

Hypothesis classes. For $d = 1, 2, \ldots, 15$, let

$$\mathcal{H}_d = \Big\{ x \mapsto \text{sign}\big(p(x)\big) \;:\; p(x) = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d, \quad w_0, \ldots, w_d \in \mathbb{R} \Big\}.$$

The polynomial p is *not given*: it is what you learn. Training a classifier in $\mathcal{H}_d$ means choosing the coefficients $w_0, \ldots, w_d$ from the training data so that the training error of $\text{sign}(p(x))$ is (approximately) as small as possible. The classes are nested, $\mathcal{H}_1 \subset \mathcal{H}_2 \subset \cdots \subset \mathcal{H}_{15}$, as required by Structural Risk Minimization.

How to fit p in practice. Map each x to the feature vector $\phi_d(x) = (x, x^2, \ldots, x^d)$ and fit a linear classifier on these features; the classifier's intercept plays the role of w_0, and its decision function *is* $p(x)$. Minimizing the 0–1 training error directly is computationally hard, so use a surrogate: logistic regression with weak regularization. This is what "approximately minimize" means. For d close to 15 the raw powers x^k are numerically ill-conditioned[1] ; you may instead use Legendre polynomials[2], which span *exactly* the same class $\mathcal{H}_d$:

```
import numpy as np
from numpy.polynomial.legendre import legvander
from sklearn.svm import LinearSVC

def features(x, d):
    return legvander(x, d)[:, 1:] # columns P_1(x)..P_d(x); constant dropped

clf = LinearSVC(C=1e4, max_iter=200000).fit(features(x_tr, d), y_tr)
p_test = clf.decision_function(features(x_te, d)) # this is the learned p(x)
y_hat = np.where(p_test > 0, 1, -1) # sign(p(x))
```

(a) Argue that $\text{VC}(\mathcal{H}_d) = d + 1$. *Hint:* a nonzero polynomial of degree at most d has at most d real roots, so $\text{sign}(p(x))$ changes sign at most d times along the real line. Which labeling of $d + 2$ sorted points needs $d + 1$ sign changes? Conversely, how would you choose the roots of p to realize any labeling of $d + 1$ points?

(b) For each $d = 1, \ldots, 15$, fit a classifier in $\mathcal{H}_d$ and record the training error $R_{\text{emp}}(d)$, the VC bound $R_{\text{emp}}(d) + f_m(d + 1)$ with $m = 200$ and $\eta = 0.05$, and the test error.

[1] tiny changes in the input may lead to massive difference in the output

[2] I include this just for general knowledge; you do not really need to know this topic.https://en.wikipedia.org/wiki/Legendre_polynomials.

(c) Plot the three curves against d on one figure. Which degree does SRM select (the one minimizing the VC bound)? Which degree does 5-fold cross-validation[3] on the training set select (`sklearn.model_selection.cross_val_score`)? Which degree minimizes the test error? Briefly explain any differences.

(d) Comment on the numerical value of the bound. Is it informative (below 0.5, the error of random guessing) for any d? What does this say about using (1) quantitatively versus qualitatively?

(e) Run a 1-nearest-neighbour classifier (`KNeighborsClassifier(n_neighbors=1)`) on the same data and report its training and test error. Reconcile the result with the fact that 1-NN has infinite VC dimension, so (1) says nothing about it.

Deliverables for Part I. Code, the requested plots and tables, and a short written discussion for every item that asks a question.

Part II: Essay (15 points)

Write an essay of **600-800 words** on *one* of the following prompts. The essay should be in your own words (NO use of AI), make precise use of the concepts from class, and cite any external sources you use (at least two, of which at least one must be a peer-reviewed paper or textbook).

Prompt A. Reading Theorems 1 and 2 together. The lecture notes state that for a hypothesis space of finite VC dimension, (Theorem 1) for every distribution there is a hypothesis with small risk with high probability, while (Theorem 2) for every hypothesis there is a distribution for which small risk is not guaranteed. Explain both results informally, why they are not contradictory, and why they motivate approaches, such as Support Vector Machines and margin-based bounds, that seek a feature space in which the data distribution is "good."

Prompt B. Infinite VC dimension does not mean poor performance. Using 1-NN, the one-parameter family $\operatorname{sign}(\sin(ax))$, and modern over-parameterized neural networks as examples, discuss the limits of VC theory as an explanation of generalization. What alternatives (margin bounds, PAC-Bayes, stability, algorithm-dependent analyses) attempt to fill the gap?

Prompt C. Is PAC learning realistic? The notes observe that fixing ε and δ and then computing m is often unrealistic, since datasets come "as is" and the resulting sample sizes are very large. Argue for or against the practical value of PAC/VC bounds, drawing on your own results from Part I. What do the bounds tell a practitioner even when the numbers are vacuous?

Prompt D. PAC-Bayes versus VC. Compare the PAC-Bayesian bound (Theorem 3 in the notes), which depends on a Kullback–Leibler divergence between a prior and a posterior over hypotheses, with the VC bound (1). What quantity measures "complexity" in each? Why can PAC-Bayes bounds be non-vacuous for large models where VC bounds are not?

[3] https://en.wikipedia.org/wiki/Cross-validation_(statistics)#

Essay rubric.

Criterion	Points
Technical accuracy and correct use of definitions	5
Depth of argument and use of concrete examples	4
Organization and clarity of writing	3
Use and citation of sources	3
Total	15

Bonus (optional, up to 6 points)

B1. Necessary versus sufficient sample size. A lower bound in the notes (Theorem 2) implies that when $h > 1 + 32\ln(1/\delta)$, about $m \approx \frac{h-1}{32\varepsilon}$ examples are *necessary*. For $\delta = 0.02$, compute the threshold on h; then for $h = 127$ and $\varepsilon = 0.01$ compute m. Compare with the *sufficient* sample size obtained by solving $f_m(127) = 0.01$ numerically, with $\eta = \delta = 0.02$. Comment on the gap.

B2. The notebook classifier. Implement the "notebook" classifier from the notes (it memorizes the labels of the first h training points and predicts a constant label everywhere else) on balanced random labels. Show empirically that its true risk is 0.5 while its empirical risk is 0 when $m = h$, and check numerically that the VC bound (1) with $h = m$ is never violated.